

WEEK 2 · 120-MINUTE CORE LECTURE

SDLC: Engineering the whole software life

SWEBOK V4 foundations · process choices · quality
evidence · responsible AI integration.

STUDYMATE INCIDENT

0-10 MIN

The code works. Did we succeed?

AI Study Plan

The demo runs. Unit tests pass. The team releases the feature.

- ! Calendar deadlines use the wrong time zone.
- ! Plans exceed the student's available hours.
- ! Private data reaches an AI API without clear consent.
- ! No fallback exists when the AI service fails.

Class vote: successful · partly successful · unsuccessful — defend your answer.

BY THE END

Four decisions you should make better

SCOPE

See the whole life

Concept through retirement—
not code alone.

PROCESS

Choose for context

Match the model to uncertainty
and consequence.

EVIDENCE

Build quality in

Define tests and gates before
release.

AI

Govern autonomy

Delegate work while retaining
accountability.

WHY SOFTWARE ENGINEERING

10-25 MIN

Programming ≠ Software Engineering

PROGRAMMING

Make code work

Algorithms · syntax · functions · local correctness.



SOFTWARE ENGINEERING

Make systems succeed

People · requirements · design · quality · security · delivery · operations · evolution.



VS

Code is an artifact. Sustainable stakeholder value is the outcome.

HARDWARE SCALED · METHODS DID NOT

Doghouse thinking cannot build a skyscraper

SMALL EXPERIMENT

Doghouse

One person · low consequence ·
improvisation may survive.

SOFTWARE CRISIS

Complex systems

Budget overruns · late delivery ·
unreliable products · unmet needs.

ENGINEERING RESPONSE

Skyscraper

Requirements · architecture ·
inspections · agreements · operations.

StudyMate at scale: thousands of users + personal data + suppliers + institutional responsibility.

SWEBOK V4 MENTAL MODEL

25-45 MIN

The life cycle is the whole lifetime

“Evolution of a system, product, service, project or other human-made entity from conception through retirement.”

SWEBOK Guide V4.0a · Chapter 10

ideation

development

production

operation

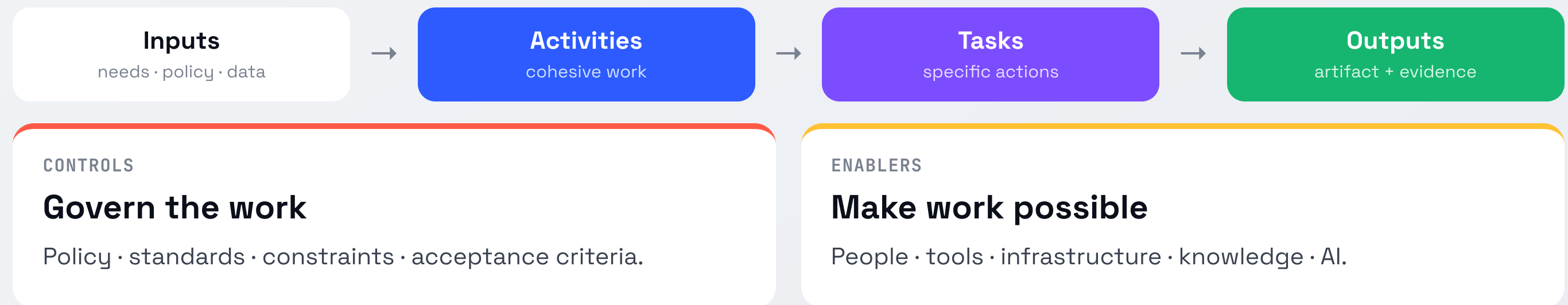
support

retirement

Deployment is a transition—not the finish line.

SWEBOK PRINCIPLE · TRANSFORMATION

A process transforms inputs into outcomes



A COMPLETE SDLC

More than “requirements → code → test”

TECHNICAL

Engineer

Requirements · design ·
build · verify · operate ·
retire.



TECHNICAL MANAGEMENT

Control

Plan · risk · configure ·
measure · assure quality.



ORGANIZATIONAL ENABLING

Enable

People · infrastructure ·
knowledge · portfolio.



AGREEMENT

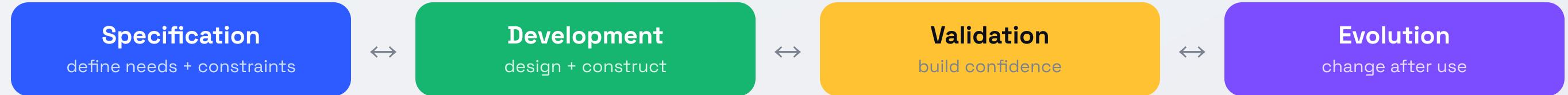
Commit

Acquisition · supply ·
customer and vendor
obligations.



SOMMERVILLE · FOUR FUNDAMENTAL ACTIVITIES

Every model must organize four kinds of work



USE THIS VIEW TO ASK

What work must occur?

All engineered software needs definition, creation, checking, and change.

DO NOT CONFUSE

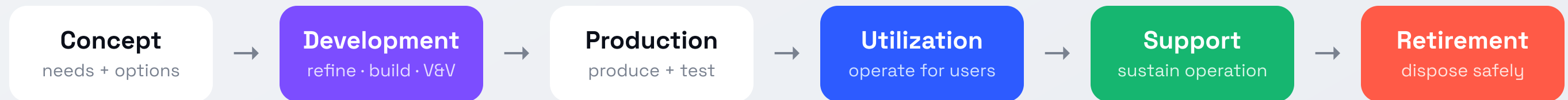
Different abstraction levels

This four-activity teaching view is not the same taxonomy as the four SWEBOOK/12207 process categories.

Waterfall, incremental, and Scrum differ mainly in timing, overlap, feedback, and control—not in whether this work exists.

SWEBOK V4 · GENERAL STAGES

Conception through retirement



Important: these are general stages—not a mandatory one-way sequence.

ONE FEATURE · WHOLE LIFE

45-75 MIN

StudyMate: AI Study Plan

The promise

Turn courses, deadlines, available hours, and priorities into a realistic seven-day study plan.

We will carry this same feature through every SDLC decision.

-  **Users:** students and instructors
-  **Data:** calendar, deadlines, preferences
-  **Suppliers:** calendar and AI APIs
-  **Risk:** privacy, wrong plans, service failure

STUDYMATE · CONCEPT & FEASIBILITY

Should we build it?

VALUE

Real problem?

Do students miss deadlines because planning is difficult?

VIABILITY

Worth the cost?

Do benefit, AI cost, schedule, and support justify investment?

FEASIBILITY

Can we do it safely?

Calendar access, privacy, AI reliability, skills, and infrastructure.

Decision

proceed

prototype

revise

stop

FROM WISH TO TESTABLE CONTRACT

What does “a good study plan” mean?

VAGUE

“Generate a good study plan.”

Good for whom? Which constraints? How fast? How verified?

TESTABLE

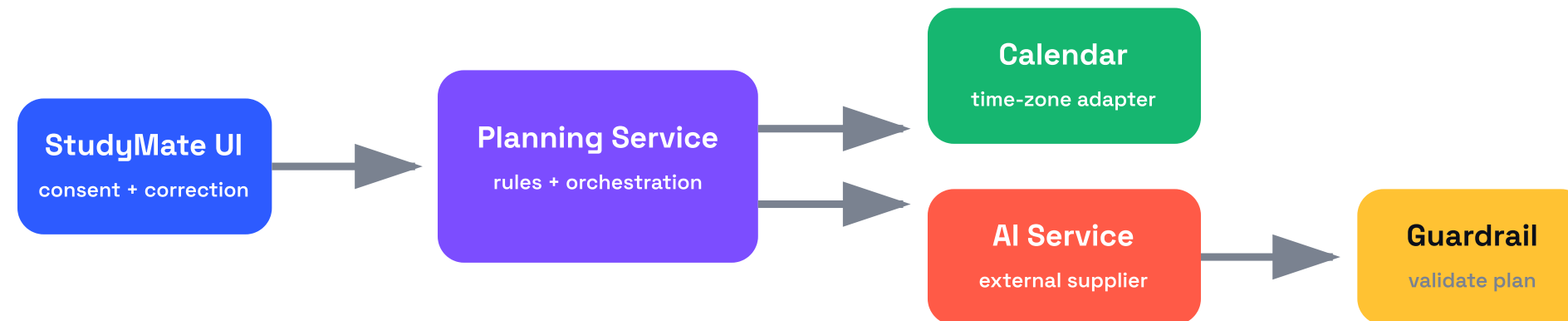
“Generate a seven-day plan within five seconds.”

No class overlap · no deadline omission · no excess over declared study hours.

Add quality constraints: consent · correction · explanation · safe failure when AI is unavailable.

TRANSLATE “WHAT” INTO “HOW”

Architecture makes risk visible



Construction evidence: reviewed code · controlled dependencies · reproducible build · requirement traceability.

BUILD CONFIDENCE THROUGH EVIDENCE

Verification and validation

UNIT

Rules

No overlap and no excess study hours.

INTEGRATION

Interfaces

Calendar time zones and AI fallback work.

SYSTEM + PRIVACY

Whole product

Performance, consent, minimization, deletion.

ACCEPTANCE

User value

Students understand and trust the plan.

Verification: built right? **Validation:** right thing?

MOST OF THE PRODUCT LIFE HAPPENS AFTER BUILD

Deploy → Utilize → Support → Retire

TRANSITION

Release safely

Pilot group · migration ·
health check · rollback.



UTILIZATION

Observe use

Latency · failures ·
rejected plans · AI cost.



SUPPORT

Sustain value

Incidents · feedback ·
model/API changes · fixes.



RETIREMENT

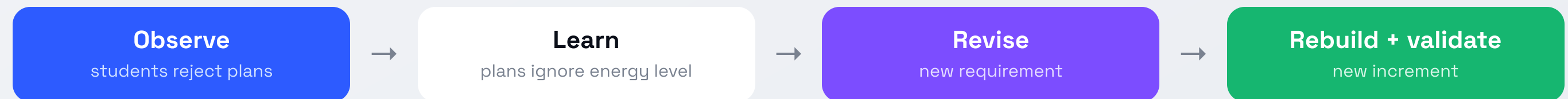
End responsibly

Revoke integrations ·
migrate · delete retained
data.



STAGES ARE NOT STRICTLY SEQUENTIAL

Operation creates new requirements



A life cycle defines meaningful stages and transitions for its stakeholders—not a universal conveyor belt.

CHOOSE THE PROCESS, NOT THE RELIGION

75-95 MIN

There is no universally ideal process

Required processes

standards + obligations

+

Model family

predictive · adaptive · risk-driven

+

Context

product · people · risk · regulation

→

Project life cycle

tailored + documented

Select · adapt · apply · measure · improve.

THREE USEFUL FAMILIES

Organize work around the dominant uncertainty

PREDICTIVE

Plan and trace

Stable constraints · strong evidence · formal transitions.

Waterfall

V-Model

ADAPTIVE

Deliver and learn

Evolving needs · fast feedback · useful increments.

Incremental

Scrum

DISCOVERY / RISK-DRIVEN

Test assumptions

Novel technology · unclear behavior · major risk first.

Prototype

Spiral

SAME WORK · DIFFERENT RHYTHM

Waterfall, incremental, and Scrum

PREDICTIVE

Waterfall

Complete and gate major phases in sequence.

SPEC → DEV → V&V → USE

ADAPTIVE DELIVERY

Incremental

Repeat the work to deliver growing usable versions.

V1 → V2 + feedback → V3 + value

ADAPTIVE FRAMEWORK

Scrum

Use a time-boxed Sprint to inspect, adapt, and create an Increment.

BACKLOG → SPRINT ↻ → INCREMENT

Selection question: which rhythm exposes this project's most important uncertainty soon enough?

CONTEXT → CHOICE → REASON

Which model family fits?

SCENARIO

Medical pump

Safety-critical · regulated · traceability.

LIKELY FIT

Predictive / V

Map each requirement to formal verification.

DECISION EVIDENCE

Failure consequence · stable constraints · audit obligations.

SCENARIO

StudyMate

Needs evolve through student feedback.

LIKELY FIT

Incremental / Scrum

Release a bounded capability and learn.

DECISION EVIDENCE

Fast feedback · useful increments · changing priorities.

SCENARIO

Novel AI advice

Behavior and technical feasibility unclear.

LIKELY FIT

Prototype / Spiral

Reduce the most dangerous uncertainty first.

DECISION EVIDENCE

Unknown quality · model risk · learning before scale.

ONE DEFECT · GROWING BLAST RADIUS 95-110 MIN

“Deadline” has no specified time zone

REQUIREMENTS

Edit one criterion

Define UTC storage and local display.

DESIGN

Change representation

Update data model and interfaces.

INTEGRATION

Rework modules

Adapter · service · fixtures · tests.

PRODUCTION

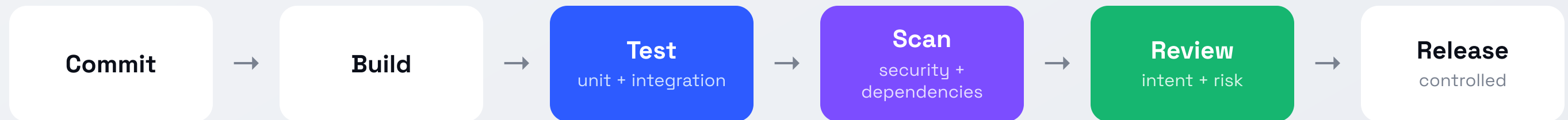
Manage incident

Wrong plans · users affected · data repair.

Shift-left = create evidence while defects are cheaper and less connected.

AUTOMATED EVIDENCE + HUMAN JUDGMENT

CI/DevSecOps as a quality gate



Failing evidence blocks unsupported changes; tools do not replace engineering judgment.

AI-ENHANCED SDLC

110-120 MIN

AI accelerates work inside the process

DISCOVER

Feedback → requirements

Summarize interviews, detect ambiguity, propose acceptance criteria.

VERIFY

Changes → evidence

Draft tests, prioritize risk, classify failures, inspect dependencies.

CREATE

Design → code

Generate alternatives, scaffolds, explanations, and repetitive transformations.

OPERATE

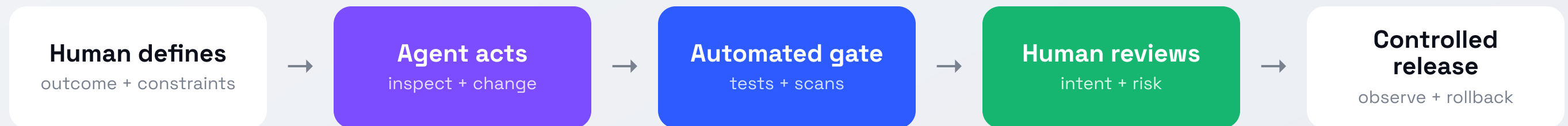
Signals → response

Detect anomalies, summarize incidents, and propose remediation.

Boundary: AI proposes and executes bounded work; humans own intent, authority, and risk.

DELEGATE AN OUTCOME · RETAIN CONTROL

A bounded agent workflow



Permissions, scope, evidence, stopping conditions, and rollback are part of the process design.

BEFORE ACCEPTING AI-GENERATED WORK

Four questions at the human gate

INTENT

Right problem?

Does this solve the approved requirement?



EVIDENCE

What proves it?

Which tests and analyses support the claims?



RISK

What can break?

Architecture · privacy · security · operations.



OWNERSHIP

Who approves?

A named human owns the decision and outcome.



Attendance challenge — intentionally false: “If AI-generated code passes automated tests, it may be merged and deployed without human review because the AI is responsible for the outcome.”

TEAM EXIT TICKET · STUDYMATE

Design a mini-SDLC in four boxes

01 · REQUIREMENT

One testable statement

Behavior + measurable condition.

03 · EVIDENCE

Gate before merge

Tests, analysis, and acceptance evidence.

02 · MODEL

Choice + reason

Risk · change · feedback · regulation.

04 · APPROVAL

Named human owner

Who accepts risk and authorizes release?

Takeaway: process structures the work; evidence supports decisions; humans own outcomes.

APPENDIX · OPTIONAL

1968: “Software Engineering” as a challenge

Software production needed the discipline, measurement, and professional practices associated with engineering.

NATO Software Engineering Conference · historical context

DECOMPOSE

Break complex systems into manageable work.

MEASURE

Make progress, quality, and risk visible.

STANDARDIZE

Use repeatable processes and shared knowledge.

APPENDIX · REFERENCE

Seven familiar process models

PREDICTIVE

Waterfall

Sequential phases and gates.

TRACEABILITY

V-Model

Development paired with tests.

ADAPTIVE

Incremental

Successive working versions.

FEEDBACK

Agile / Scrum

Time-boxed adaptive delivery.

DISCOVERY

Prototype

Learn by building a model.

RISK-DRIVEN

Spiral

Address major risk each cycle.

MINIMAL

Big Bang

Little planning; high uncertainty.

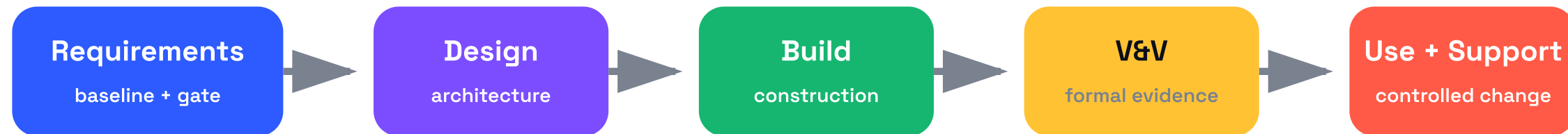
RULE

Context wins

Tailor stages, gates, and evidence.

APPENDIX · PREDICTIVE MODEL

Waterfall: major phases and formal gates



Strong traceability and milestones · feedback to earlier baselines may be costly

USEFUL WHEN

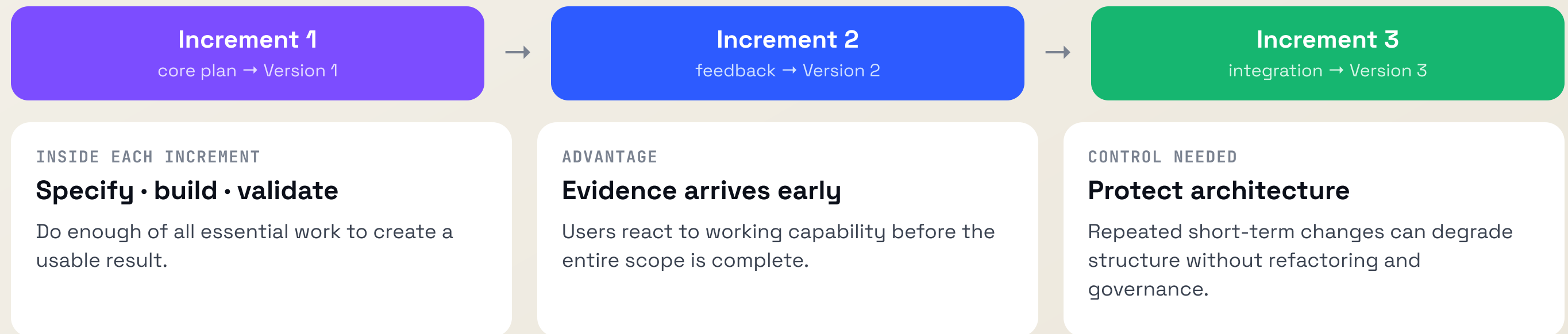
Constraints are stable, evidence is formal, and transitions require approval.

MAIN RISK

Wrong assumptions can survive until working software and user feedback arrive late.

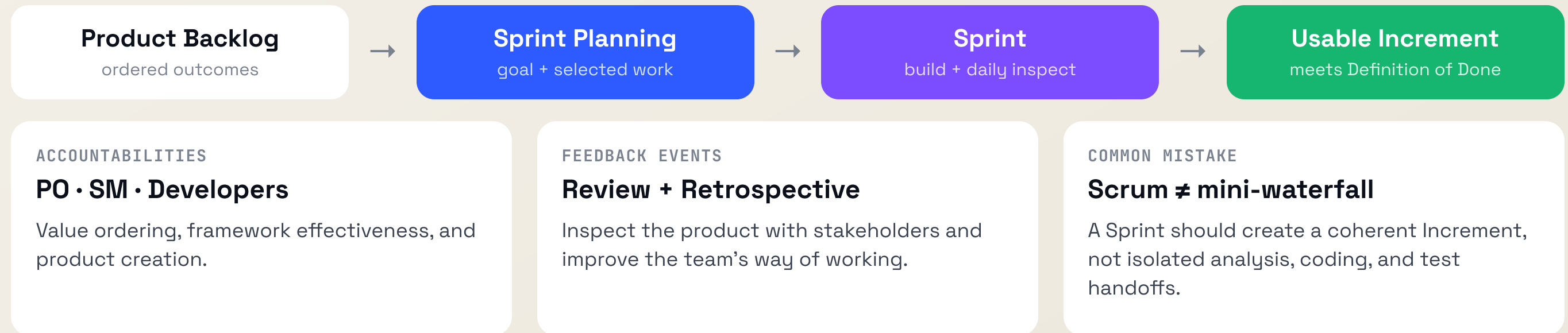
APPENDIX · ADAPTIVE DELIVERY

Incremental: repeat the work, grow the product



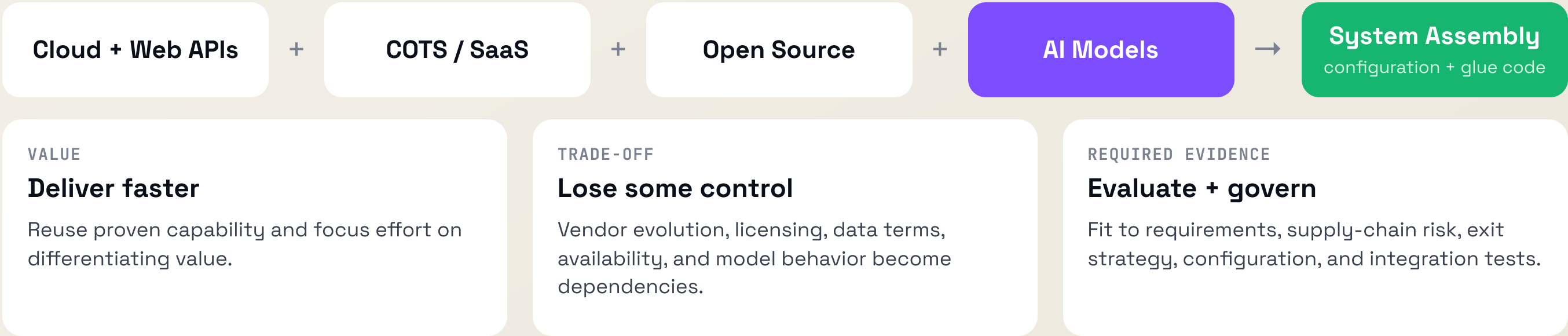
APPENDIX · ADAPTIVE FRAMEWORK

Scrum: a rhythm for inspection and adaptation



APPENDIX · BUILD, BUY, OR REUSE

Modern systems are assembled as well as coded



StudyMate: calendar and AI services are architecture decisions, supplier relationships, and continuing risks—not free implementation details.

APPENDIX · DEVSECOPS

Security tooling by evidence type

SAST

Source evidence

Analyze code without running the application.

SCA

Supply-chain evidence

Dependencies, vulnerabilities, and licenses.

SECRETS

Credential evidence

Block exposed keys, tokens, and passwords.

DAST

External runtime evidence

Attack a running test application.

IAST

Internal runtime evidence

Connect behavior to source locations.

LIMIT

Tools ≠ judgment

Threat modeling and risk ownership remain human.

APPENDIX · VERIFY BEFORE CITING

Reported AI industry signals

GARTNER

70%

application design and development reportedly influenced by AI.

MCKINSEY

2×

reported productivity; over 40% production code AI-assisted.

QODO 2025

81%

reported quality outcome for AI-assisted review.

ATLASSIAN 2026

38.7%

agent comments reportedly triggered additional fixes.

Do not cite this slide: verify original report, sample, metric definition, and method.

APPENDIX · SOURCE-CHECK REQUIRED

Reported technical-debt pressure

IT BALANCE SHEETS

≈40%

reportedly associated with technical debt.

PROJECT PREMIUM

10–20%

reportedly paid because of poor architecture.

CAPACITY

≈25%

time and budget reportedly spent managing debt.

Instructional principle: generation speed without understanding and review accelerates debt.

APPENDIX · REFERENCE

AI opportunities by SDLC phase

REQUIREMENTS

Mine + clarify

Feedback analysis, stories, ambiguity detection.

DESIGN

Generate + simulate

Options, bottlenecks, anti-patterns, trade-offs.

DEVELOPMENT

Implement + explain

Multi-file changes, refactoring, documentation.

TESTING

Create + prioritize

Edge cases, risk-weighted tests, failure analysis.

DEPLOYMENT

Predict + configure

Pipeline failures, IaC, rollback support.

MAINTENANCE

Detect + triage

Anomalies, incident summaries, remediation drafts.

APPENDIX · FULL REVIEW GATE

Seven questions before acceptance

01

Intent

Right approved requirement?

02

Understanding

Can a human explain it?

03

Evidence

What proves the claims?

04

Architecture

Does it preserve boundaries?

05

Security

New data or permission risk?

06

Operations

Observable and reversible?

07

Accountability

Who approves and owns it?

DECISION

Accept · revise · reject

Never merge on model reputation.

APPENDIX · PRIMARY FRAMEWORK

References and instructor notes

Primary source

SWEBOK Guide V4.0a, Chapter 10: Software Engineering Process, especially §§1.2 and 2.1–2.11.

Standards context

ISO/IEC/IEEE 12207 terminology informs the four life-cycle process categories.

Lecture notes: ../Intro2SE-Week02-SDLC-AI-Foundations.md

 Core lecture: slides 1–27

 Appendix: optional reference only

 Verify all external quantitative claims before citation